DOKUMENTASI KODE — ENTRY POINT TRANSMITTER

Entry Point Transmitter

receiverArduino.py — Peluncur GUI / CLI Panel Kontrol (Raspberry Pi / PC)

Judul Skripsi	Pengembangan Robot Akuatik untuk Feeder Otomatis di Tambak
Sub-sistem	Transmitter / Kendali (Raspberry Pi + NRF24L01)
Berkas	Master Code/Transmitter/receiverArduino.py
Bahasa	Python 3
Tanggal	25 July 2026

Dokumentasi Kode — Entry Point Transmitter `receiverArduino.py`

Dokumen ini menjelaskan berkas `receiverArduino.py` , yaitu **satu-satunya titik masuk (entry point)** sisi *transmitter* (Raspberry Pi / PC) untuk mengendalikan robot feeder akuatik. Menjalankan file ini akan menjalankan seluruh sistem kendali: GUI, telemetri, dan logging.

> Dokumentasi arsitektur GUI yang lebih mendalam (analisis kebutuhan, perancangan, pengujian) ada pada `Dokumentasi_GUI_Feeder_SDLC.pdf` di folder yang sama. Dokumen ini fokus pada **kode** `receiverArduino.py` dan modul yang dipanggilnya.

---

1. Gambaran Umum

`receiverArduino.py` berperan sebagai **peluncur (launcher)** dengan dua mode operasi:

Mode	Cara jalan	Kegunaan
GUI (default)	`python receiverArduino.py`	Panel kontrol grafis (CustomTkinter) — dipakai operator di lapangan
CLI (lama)	`python receiverArduino.py --cli`	Mode teks sederhana — untuk uji cepat / tanpa layar sentuh

Filosofi desainnya: **logika inti tidak ditulis di sini**. Berkas ini hanya memilih mode lalu mendelegasikan pekerjaan ke modul lain:

- `feeder_gui.py` — antarmuka grafis (kelas `FeederGUI`).
- `feeder_core.py` — mesin di baliknya: `RadioLink` , `MissionEstimator` , `ActivityLogger` , `MPUSerialReader` .

Dengan begitu, `receiverArduino.py` tetap ringkas dan mudah dibaca sebagai peta seluruh sistem.

---

2. Struktur Berkas

Hanya ada tiga fungsi + satu *guard*:

Fungsi	Baris	Peran
`run_gui()`	25–39	Membuka panel kontrol grafis
`run_cli()`	46–109	Menjalankan antarmuka teks lama
`main()`	114–118	Memilih mode dari argumen baris perintah
`if __name__ == "__main__"`	121–122	Titik eksekusi saat file dijalankan langsung

---

3. Pemilihan Mode — `main()`

Baris 114–118. Sangat sederhana: bila argumen `--cli` atau `-c` ada di `sys.argv` , jalankan mode teks; jika tidak, buka GUI.

```
def main():
if "--cli" in sys.argv or "-c" in sys.argv:
run_cli()
else:
run_gui()
```

4. Mode GUI — `run_gui()`

Baris 25–39. Mengimpor `FeederGUI` dari `feeder_gui.py`, lalu memanggil `FeederGUI().mainloop()` untuk membuka jendela.

Bagian penting adalah **penanganan** `ImportError`: bila pustaka GUI (mis. `customtkinter`) belum terpasang, program tidak *crash* dengan *traceback* membingungkan, melainkan mencetak petunjuk pemasangan yang jelas lalu keluar rapi:

```
try:
from feeder_gui import FeederGUI
except ImportError as e:
print("Gagal memuat GUI:", e)
print(" pip install customtkinter (wajib utk GUI)")
print(" pip install pyrf24 pyserial (utk NRF & MPU nyata di Pi)")
print(" python receiverArduino.py --cli (mode teks lama)")
sys.exit(1)
```

> **Catatan pemasangan (penting di Windows/PC dengan banyak Python):** pesan `customtkinter belum terpasang` muncul bila paket terpasang di interpreter Python yang **berbeda** dari yang menjalankan skrip. Pastikan memasang ke interpreter yang sama, mis. `& "C:\path\ke\python.exe" -m pip install customtkinter pyserial`. Di PC (tanpa NRF), `pyrf24` boleh dilewati — GUI otomatis masuk **mode simulasi**.

Mode GUI otomatis memakai **NRF asli** bila dijalankan di Raspberry Pi (pustaka `pyrf24` tersedia), dan **mode simulasi** bila tidak — sehingga panel tetap bisa didemokan di PC tanpa perangkat keras.

---

5. Mode CLI — `run_cli()`

Baris 46–109. Berbeda dengan GUI, **seluruh logika mode teks ada di dalam fungsi ini**, jadi paling banyak dijelaskan di sini. Alurnya:

5.1 Inisialisasi radio (baris 47–63)

```
radio = RF24(24, 0) # CE=GPIO24, CSN=CE0 (pin Raspberry Pi)
radio.begin()
radio.open_tx_pipe(b"00001") # alamat = "00001"
radio.setPALevel(RF24_PA_LOW)
radio.setDataRate(RF24_250KBPS)
radio.setChannel(0x76)
```

> Parameter radio (**alamat** `00001`, **channel** `0x76`, **data rate** `250KBPS`, **PA** `LOW`) **harus identik** dengan firmware robot `Finalnew.ino`; jika berbeda, kedua modul tidak akan terhubung. Perhatikan pin CE/CSN di Raspberry (`24, 0`) berbeda dari ESP32 (`4, 5`) karena beda papan.

5.2 Heartbeat otomatis (baris 65–79)

Sebuah *thread* latar (`daemon`) mengirim paket `PING` tiap **0,5 detik** selama sebuah misi berjalan (`state["mission_running"] == True`). Ini menjaga *failsafe* robot tetap "hidup" (robot berhenti bila tak ada sinyal > 3 detik).

```
def heartbeat_loop():
while True:
if state["mission_running"]:
radio_send(b"PING")
time.sleep(HEARTBEAT_INTERVAL_S) # 0.5 s
threading.Thread(target=heartbeat_loop, daemon=True).start()
```

Akses ke `radio.write()` dilindungi `threading.Lock` (fungsi `radio_send`) agar *thread* input utama dan *thread* heartbeat tidak menulis ke radio bersamaan.

5.3 Loop perintah operator (baris 81–108)

- Operator mengetik **Panjang Tambak**. Bila mengetik `STOP` , program menghentikan heartbeat (`mission_running=False`) dan mengirim paket `STOP` sebanyak 5 kali (perulangan untuk keandalan).
- Selain itu, operator mengetik **Lebar Tambak**, lalu program menyusun payload `P=<panjang>;L=<lebar>;MODE=AUTO` dan mengirimnya 5 kali. Bila salah satu pengiriman berhasil -> tampilkan `SUCCESS` dan set `mission_running=True` (heartbeat mulai).

Mengapa dikirim 5 kali? NRF24 kadang gagal pada pengiriman pertama; mengulang beberapa kali meningkatkan peluang paket diterima tanpa perlu protokol *retry* rumit.

---

6. Konsistensi Protokol dengan Firmware

Paket yang dikirim `receiverArduino.py` harus dipahami oleh `Finalnew.ino` :

Paket dikirim	Oleh	Ditangani firmware
`P=<m>;L=<m>;MODE=AUTO`	input operator	Parsing dimensi -> jalankan misi zigzag
`PING`	thread heartbeat (0,5 s)	Segarkan *heartbeat*, cegah *failsafe*
`STOP`	perintah `STOP` operator	Batalkan misi / matikan motor

---

7. Peta Modul (Arsitektur)

`receiverArduino.py` adalah lapisan tertipis; kekuatan sistem ada di modul yang dipanggilnya (khususnya via mode GUI):

Berkas	Isi utama	Tanggung jawab
`receiverArduino.py`	`run_gui` , `run_cli` , `main`	Entry point & pemilihan mode
`feeder_gui.py`	kelas `FeederGUI`	Presentasi: input, tombol, gauge, LED, panel MPU, tampilan log
`feeder_core.py`	`RadioLink`	Abstraksi NRF24 (`pyrf24`) + *auto-fallback* `MockRadio` (simulasi)
	`plan_coverage()` / `CoveragePlan`	Terjemahkan tambak persegi + titik awal + lebar lintasan -> daftar segmen "maju x / belok y°" (path zigzag) + `encode_mission()`
	`MissionEstimator`	Estimasi fase/PWM/RPM/kecepatan sepanjang timeline zigzag
	`ActivityLogger`	Pencatatan aktivitas berstempel waktu + ekspor CSV/TXT
	`MPUSerialReader`	Baca data MPU6050 (roll/pitch/yaw) via serial USB

Perbedaan penting dengan CLI: mode GUI kini menyediakan **keypad numerik** (layar sentuh tanpa keyboard), input **titik awal**, **lebar lintasan**, dan **kecepatan**, plus **preview jalur** (kanvas + daftar langkah maju/belok). GUI mengirim paket **diperluas** `P=..;L=..;MODE=AUTO;SPD=..;LW=..;DIR=..` , sedangkan mode CLI di berkas ini masih mengirim bentuk dasar `P=..;L=..;MODE=AUTO` (tetap valid —

robot memakai default untuk field yang tak ada). GUI juga menampilkan **telemetri estimasi** dan **data MPU6050 nyata** via serial USB; CLI hanya mengirim perintah tanpa telemetri.

---

8. Cara Menjalankan

```
cd "Master Code/Transmitter"
python receiverArduino.py # buka GUI (default)
python receiverArduino.py --cli # mode teks lama
```

Dependensi (lihat `requirements.txt`):

Paket	Untuk	Wajib di
`customtkinter`	framework GUI	GUI (PC & Pi)
`pyserial`	baca MPU6050 via serial USB	GUI (opsional di PC)
`pyrf24`	driver NRF24L01	**hanya Raspberry Pi** (dilewati di Windows)

Di **PC** cukup `pip install customtkinter pyserial` — NRF otomatis disimulasikan. Di **Raspberry Pi** tambahkan `pyrf24` untuk kendali radio nyata. Selalu pasang ke interpreter Python yang sama dengan yang menjalankan skrip.

---

9. Ringkasan

`receiverArduino.py` sengaja dijaga **minimal dan jelas**: satu entry point, dua mode, penanganan error yang ramah, dan pendelegasian bersih ke `feeder_gui.py` / `feeder_core.py`. Struktur ini membuat sistem mudah dipakai (satu perintah untuk semua) sekaligus mudah dikembangkan — penambahan fitur cukup dilakukan di modul core/GUI tanpa mengubah titik masuk.